EXPERIMENT 5: - INFORMATION EXTRACTION USING SPACY

Learning Objective

To extract useful information from customer reviews using spaCy NLP techniques.

Problem Statement

To analyze customer reviews and extract tokens, lemmas, POS tags, and named entities automatically.

Tasks to Perform

1. Load the Amazon Fine Food Reviews dataset.
2. Clean missing reviews and remove duplicates.
3. Install and load the spaCy English model.
4. Extract tokens, lemmas, and POS tags from five reviews.
5. Count the frequency of major POS categories.
6. Perform Named Entity Recognition (NER) on five reviews.
7. Create a function to extract tokens, lemmas, POS tags, and named entities.
8. Test the function using five customer reviews.
9. Display the extracted information in a structured format.

AIM

To extract linguistic features and named entities from customer reviews using spaCy.

ALGORITHM

1. Load the dataset and required spaCy model.
2. Remove missing reviews and duplicate records.
3. Extract tokens, lemmas, and POS tags.
4. Count the major POS categories.
5. Identify named entities using NER.
6. Display the extracted information in a structured format.

PROGRAM AND OUTPUT

```
# Question 1: Load the Amazon Fine Food Reviews dataset used in the previous experiment.
# Luei johncena (241801140)
import pandas as pd
import spacy
from collections import Counter

# Public CSV mirror of the Amazon Fine Food Reviews dataset
url = "https://huggingface.co/datasets/PJ2005/amazon-fine-food-reviews/resolve/main/Reviews.csv"
df = pd.read_csv(url, nrows=10000)
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df[["Summary", "Text"]].head())
```

```
Dataset shape: (10000, 10)
Columns: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator', 'Score', 'Time', 'Su
```

	Summary	Text
0	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	"Delight" says it all	This is a confection that has been around a fe...
3	Cough Medicine	If you are looking for the secret ingredient i...
4	Great taffy	Great taffy at a great price. There was a wid...

```
# Question 2: Clean the dataset by removing missing/null reviews and duplicate records.
# Luei johncena (241801140)
df = df.dropna(subset=["Text"]).drop_duplicates().copy()
df["Text"] = df["Text"].astype(str)
df = df.head(10000).copy()
print("Shape after cleaning:", df.shape)
print("Missing review text:", df["Text"].isna().sum())
print("Duplicate rows:", df.duplicated().sum())
```

```
Shape after cleaning: (10000, 10)
Missing review text: 0
Duplicate rows: 0
```

```
# Question 3: Install/import spaCy and load its English language model.
# Luei johncena (241801140)
!pip -q install spacy
!python -m spacy download en_core_web_sm
import spacy
nlp = spacy.load("en_core_web_sm")
print("spaCy English model loaded.")
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 90.4 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
spaCy English model loaded.
```

```
# Question 4: Apply spaCy preprocessing to five reviews; display tokens, lemmas, and POS tags, and identify POS categories
# Luei johncena (241801140)
def analyze_review(text):
    doc = nlp(str(text))
    return {
        "Tokens": [t.text for t in doc if not t.is_space],
        "Lemmas": [t.lemma_ for t in doc if not t.is_space],
        "POS_Tags": [(t.text, t.pos_) for t in doc if not t.is_space],
        "POS_Categories": sorted(set(t.pos_ for t in doc if not t.is_space))
    }

for i, text in enumerate(df["Text"].head(5), 1):
    result = analyze_review(text)
    print(f"Review {i}")
    print("Tokens:", result["Tokens"][:30])
    print("Lemmas:", result["Lemmas"][:30])
    print("POS tags:", result["POS_Tags"][:30])
    print("POS categories:", result["POS_Categories"])
    print("-" * 50)
```

```
Review 1
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fou
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'VERB']
--------------------------------------------------
Review 2
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually',
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'smal
POS tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PRO
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 3
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', 'It', 'is', 'a', '
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', 'it', 'be', 'a', 'lig
POS tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been',
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 4
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have',
POS tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret',
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 5
Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taf
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy
POS tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PU
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'DET', 'NOUN', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
```

```
# Question 5: Count and display the frequency of major POS categories in the reviews.
# Luei johncena (241801140)
pos_counts = Counter()
for doc in nlp.pipe(df["Text"].astype(str).tolist(), batch_size=64):
    pos_counts.update(token.pos_ for token in doc if not token.is_space)

print("POS category frequencies:")
display(pd.DataFrame(pos_counts.most_common(), columns=["POS Category", "Frequency"]))
```

```
POS category frequencies:
```

	POS Category	Frequency
0	NOUN	143585
1	PRON	100147
2	PUNCT	100048
3	VERB	94878
4	ADJ	71588
5	ADP	68797
6	DET	68305
7	AUX	56831
8	ADV	50405
9	PROPN	37863
10	CCONJ	32436
11	PART	24871
12	SCONJ	16460
13	NUM	11564
14	X	3029
15	SYM	2831
16	INTJ	1729

```
# Question 6: Apply spaCy Named Entity Recognition (NER) to five reviews and extract entity text and labels.
# Luei johncena (241801140)
for i, text in enumerate(df["Text"].head(5), 1):
    doc = nlp(text)
    entities = [(ent.text, ent.label_) for ent in doc.ents]
    print(f"Review {i} entities:", entities)
```

```
Review 1 entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
Review 2 entities: [('Jumbo Salted', 'ORG'), ('Jumbo', 'WORK_OF_ART')]
Review 3 entities: [('around a few centuries', 'DATE'), ('pillowy citrus gelatin', 'ORG'), ('Filberts', 'GPE'), ("C.S. Lewis
```

```
Review 4 entities: [('Robitussin', 'GPE'), ('the Root Beer Extract I', 'LAW')]
Review 5 entities: [('Delivery', 'PERSON')]
```

```
# Question 7: Create a function that accepts a customer review and returns tokens, lemmas, POS tags, and named entities.
# Luei johncena (241801140)
def extract_information(review):
    doc = nlp(str(review))
    return {
        "Tokens": [t.text for t in doc if not t.is_space],
        "Lemmas": [t.lemma_ for t in doc if not t.is_space],
        "POS_Tags": [(t.text, t.pos_) for t in doc if not t.is_space],
        "Named_Entities": [(ent.text, ent.label_) for ent in doc.ents]
    }

sample = extract_information(df["Text"].iloc[0])
print("Tokens:", sample["Tokens"][:30])
print("Lemmas:", sample["Lemmas"][:30])
print("POS tags:", sample["POS_Tags"][:30])
print("Named entities:", sample["Named_Entities"])
```

```
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fou
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality
Named entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
```

```
# Question 8: Test the information extraction function using any five customer reviews.
# Luei johncena (241801140)
for i, text in enumerate(df["Text"].head(5), 1):
    result = extract_information(text)
    print(f"Review {i}")
    print("Tokens:", result["Tokens"][:15])
    print("Lemmas:", result["Lemmas"][:15])
    print("POS tags:", result["POS_Tags"][:15])
    print("Entities:", result["Named_Entities"])
    print("-" * 50)
```

```
Review 1
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fou
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality
Entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
--------------------------------------------------
Review 2
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually',
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'smal
POS tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PRO
Entities: [('Jumbo Salted', 'ORG'), ('Jumbo', 'WORK_OF_ART')]
--------------------------------------------------
Review 3
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', 'It', 'is', 'a']
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', 'it', 'be', 'a']
POS tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been'
Entities: [('around a few centuries', 'DATE'), ('pillowy citrus gelatin', 'ORG'), ('Filberts', 'GPE'), ("C.S. Lewis'", 'ORG
--------------------------------------------------
Review 4
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have',
POS tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret'
Entities: [('Robitussin', 'GPE'), ('the Root Beer Extract I', 'LAW')]
--------------------------------------------------
Review 5
Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taf
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy
POS tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PU
Entities: [('Delivery', 'PERSON')]
--------------------------------------------------
```

```
# Question 9: Display extracted information in a structured format.
# Luei johncena (241801140)
structured = []
for idx, text in enumerate(df["Text"].head(5), 1):
    result = extract_information(text)
    structured.append({
        "Review_No": idx,
        "Tokens": result["Tokens"][:15],
        "Lemmas": result["Lemmas"][:15],
        "POS_Tags": result["POS_Tags"][:15],
        "Named_Entities": result["Named_Entities"]
    })

structured_df = pd.DataFrame(structured)
display(structured_df)
```

	Review_No	Tokens	Lemmas	POS_Tags	Named_Entities
0	1	[I, have, bought, several, of, the, Vitality, ...	[I, have, buy, several, of, the, Vitality, can...	[(I, PRON), (have, AUX), (bought, VERB), (seve...	[(Vitality, ORG), (My Labrador, PERSON)]
1	2	[Product, arrived, labeled, as, Jumbo, Salted,...	[product, arrived, label, as, Jumbo, Salted, P...	[(Product, NOUN), (arrived, AUX), (labeled, VE...	[(Jumbo Salted, ORG), (Jumbo, WORK_OF_ART)]
2	3	[This, is, a, confection, that, has, been, aro...	[this, be, a, confection, that, have, be, arou...	[(This, PRON), (is, AUX), (a, DET), (confectio...	[(around a few centuries, DATE), (pillowy citr...
		[If, you, are, looking, for,	[if, you, be, look, for, the,	[(If, SCONJ), (you, PRON),	[(Robitussin, GPE), (the Root

Next steps: Generate code with structured_df | New interactive sheet

RESULT

Customer reviews were processed using spaCy to extract tokens, lemmas, POS tags, and named entities. The extracted information was displayed in a structured format.